

Testar

Ontem você conferiu no olho se não tinha quebrado nada. Hoje o computador confere por você — e não esquece de conferir de novo amanhã.

Continuação direta da refatoração.

Você precisa do jogo já refatorado — com `calcular_dano`, `usar_habilidade` e `BonusTemporario` — antes de começar.

Onde você está

Lembre de ontem. Cada um dos cinco capítulos terminava com uma caixa verde mandando você jogar de novo e conferir com os próprios olhos que nada tinha mudado. E o anexo C terminava com vinte itens de checklist, todos conferidos do mesmo jeito: abrindo o jogo, chegando na fase certa, provocando a situação certa, e olhando a tela.

Funcionou. Mas conte o custo: para conferir que o Esqueleto ainda remonta os ossos, você precisou vencer três fases antes. Para conferir que o elixir não acumula, precisou chegar no Orc, ganhar o elixir, entrar num combate e beber dois. Cada conferência dessas custa uns três minutos de jogo — e você fez isso **cinco vezes**, uma por capítulo.

Agora responda com sinceridade: você conferiu os vinte itens do checklist? Todos? Ou conferiu os que eram fáceis de chegar e confiou nos outros?

O PROBLEMA DE VERDADE

Conferir no olho não é errado — é **caro**. E o que é caro, a gente faz uma vez e depois deixa de fazer. Um teste automatizado custa caro uma vez, quando você escreve, e depois custa quase zero para sempre.

O que é um teste automatizado

Um teste é um pedaço de código que roda o seu código e **afirma** que alguma coisa é verdade. Se for, ele não diz nada. Se não for, ele grita.

É exatamente o que você fazia no olho, escrito em Python. "Curar não pode passar da vida máxima" era uma frase na sua cabeça enquanto você jogava. Hoje ela vira uma linha de código que qualquer um pode rodar em meio segundo, hoje, amanhã e daqui a seis meses.

A sua pasta

Ontem a regra era que nenhum arquivo novo podia entrar. Hoje entra **um**, e só um:

```
rpg/
  personagem.py    Personagem e BonusTemporario
  classes.py       Guerreiro, Mago e os seis inimigos
  itens.py         poções, elixires e o inventário
  main.py          fases, inventário, combate e progressão
  testes.py        NOV0: só testes, nenhuma regra de jogo
```

A REGRA DO DIA

O jogo não sabe que os testes existem. Nenhum `import testes` vai aparecer nos outros quatro arquivos. A seta aponta num sentido só: `testes.py` conhece o jogo, o jogo não conhece os testes. Se você precisar mexer no jogo para o teste funcionar, tem uma caixa no capítulo 4 sobre quando isso é legítimo.

O ponto de partida

Hoje só funciona em cima do código refatorado de ontem. Os testes deste material chamam `calcular_dano`, `usar_habilidade` e `bonus_ataque.ativo()` — três coisas que só existem depois da parte 10.

Se o seu jogo não chegou lá, pegue os quatro arquivos da pasta `Codigo Base/`. Eles são o jogo de ontem com os cinco capítulos aplicados, e servem também de gabarito da refatoração: se você quiser comparar a sua solução com uma solução possível, é ali.

O caminho

1 A primeira afirmação

Uma palavra do Python, `assert`, e a sua primeira conferência que não usa o olho.

2 Um arquivo só de testes

Com doze afirmações soltas, o primeiro erro esconde os outros onze. Entra o `unittest`.

3 Testar o que sorteia

Como afirmar qualquer coisa sobre um dano que muda toda vez.

4 Testar o que imprime

Quase tudo no seu jogo termina em `print`. E `print` não se afirma.

5 O teste que pega a regra

As duas regras que quase morreram ontem, e o teste que você tem que ver falhar.

Os capítulos estão em ordem de dificuldade e cada um depende do anterior. O capítulo 5 é o mais importante do dia: é onde o teste deixa de ser exercício e passa a pagar por si.

Como cada capítulo funciona: problema, explicação, dois exemplos completos fora do RPG, e aí o exercício no seu jogo. Os exemplos não são enfeite — testar é mais fácil de enxergar em código pequeno, que você não escreveu e não tem carinho nenhum.

Sumário

01 A primeira afirmação

02 Um arquivo só de testes

03 Testar o que sorteia

04 Testar o que imprime

05 O teste que pega a regra

AN Anexo A: erros que você vai ver

AN Anexo B: extras

AN Anexo C: checklist de entrega

A primeira afirmação

PROBLEMA → EXPLICAÇÃO → EXEMPLO → EXEMPLO → EXERCÍCIO

O problema

Abra o `personagem.py` e olhe o `curar`:

```
def curar(self, quantidade):
    if quantidade <= 0:
        return
    self._vida = min(self._vida + quantidade, self._vida_maxima)
```

Esse método tem duas regras escondidas dentro dele. A primeira: curar com número zero ou negativo não faz nada. A segunda: a cura nunca passa da vida máxima — é para isso que serve o `min`.

Como você conferiu essas duas regras até hoje? Abrindo o jogo, apanhando de propósito até ficar com pouca vida, bebendo uma poção, e olhando o número na tela. E a primeira regra, a do número negativo, você provavelmente nunca conferiu na vida, porque não existe item nenhum no jogo que cure um valor negativo. A regra está escrita ali e ninguém nunca olhou para ela.

O INCÔMODO

Existe código no seu jogo que nunca foi conferido por ninguém, e você não tem como saber qual é. Não porque você foi desleixado, mas porque conferir jogando só alcança o que dá para provocar jogando.

A explicação: `assert`

O Python tem uma palavra reservada para isso. Ela se chama `assert`, que em português é **afirmar**:

```
assert condição, "mensagem se der errado"
```

O comportamento é o mais simples possível, e é justamente por isso que ele serve:

SE A CONDIÇÃO FOR

O QUE ACONTECE

Verdadeira

Absolutamente nada. O programa segue para a linha seguinte.

Falsa

O programa para na hora com `AssertionError` e imprime a sua mensagem.

Repare no que isso significa na prática: um arquivo cheio de `assert` que roda até o fim **sem imprimir nada** é um arquivo de boas notícias. Silêncio é aprovação.

A vírgula e a mensagem são opcionais — `assert saldo == 100` funciona sozinho. Mas sem a mensagem, quando ele falhar, tudo que você vai ver é a palavra `AssertionError` e o número da linha. Com a mensagem, você lê o que era para ser verdade. **Sempre escreva a mensagem.**

COMO ESCREVER UMA AFIRMAÇÃO

Todo teste, do mais bobo ao mais sofisticado, tem a mesma forma de três partes. Ela tem nome na profissão — *arrange*, *act*, *assert* — e vale decorar:

PARTE	O QUE VOCÊ FAZ
Preparar	Cria os objetos de que o teste precisa, no estado de que ele precisa.
Agir	Chama o método que está sendo testado. Uma chamada, não cinco.
Afirmar	Diz o que tem que ser verdade depois disso.

Um teste que mistura as três partes vira ilegível, e um teste ilegível é pior que nenhum teste: quando ele falhar, você vai perder mais tempo entendendo o teste do que consertando o código.

Exemplo 1: o cofrinho

Uma classe pequena com duas regras: não dá para sacar mais do que tem, e não dá para depositar valor negativo.

O CÓDIGO A SER TESTADO

```
class Cofrinho:
    def __init__(self):
        self.saldo = 0

    def depositar(self, valor):
        if valor <= 0:
            return
        self.saldo += valor

    def sacar(self, valor):
        if valor > self.saldo:
            return 0
        self.saldo -= valor
        return valor
```

OS TESTES

```

# preparar
cofre = Cofrinho()
# agir
cofre.depositar(50)
# afirmar
assert cofre.saldo == 50, "depositar 50 num cofre vazio deveria dar saldo 50"

cofre = Cofrinho()
cofre.depositar(-20)
assert cofre.saldo == 0, "deposito negativo nao pode mexer no saldo"

cofre = Cofrinho()
cofre.depositar(30)
tirado = cofre.sacar(100)
assert tirado == 0, "nao pode sacar mais do que tem"
assert cofre.saldo == 30, "saque recusado nao pode mexer no saldo"

print("Todos os testes do cofrinho passaram.")

```

Rode. Se aparecer só a última linha, as quatro afirmações são verdadeiras. Repare em três decisões desse código, porque você vai repetir as três a aula inteira:

- **Cada teste cria um cofre novo.** Se o segundo teste usasse o cofre do primeiro, ele começaria com saldo 50 e a afirmação estaria errada. Teste que depende do teste anterior é uma armadilha — no capítulo 2 você vai ver o mecanismo que resolve isso de vez.
- **O saque recusado tem duas afirmações.** Uma sobre o que o método devolveu, outra sobre o estado que ficou. Recusar e devolver zero não basta: o cofre também não podia ter mexido no saldo. Regras diferentes, afirmações diferentes.
- **A mensagem descreve a regra, não o código.** "nao pode sacar mais do que tem" é útil daqui a um ano. "assert falhou na linha 12" não é.

Exemplo 2: o conversor de temperatura

Este exemplo existe para mostrar o que um teste pega e o olho não pega.

O CÓDIGO, COM UM ERRO DENTRO

```

def celsius_para_fahrenheit(c):
    return c * 9 / 5 + 32

def fahrenheit_para_celsius(f):
    return (f - 32) * 5 / 9

def classificar(c):
    if c < 0:
        return "congelando"
    elif c < 15:
        return "frio"
    elif c < 25:
        return "agradavel"
    elif c < 35:
        return "quente"
    return "muito quente"

```

Olhe esse código por trinta segundos e diga se ele está certo. A maioria das pessoas diz que sim. Agora os testes:

```
assert celsius_para_fahrenheit(0) == 32, "0C tem que dar 32F"
assert celsius_para_fahrenheit(100) == 212, "100C tem que dar 212F"
assert celsius_para_fahrenheit(-40) == -40, "-40 e o ponto onde as duas escalas se cruzam"

assert fahrenheit_para_celsius(32) == 0, "32F tem que dar 0C"
assert fahrenheit_para_celsius(212) == 100, "212F tem que dar 100C"

assert classificar(-5) == "congelando", "-5 esta congelando"
assert classificar(0) == "frio", "0 nao esta mais congelando"
assert classificar(14.9) == "frio"
assert classificar(15) == "agradavel", "15 e o comeco do agradavel"
assert classificar(24.9) == "agradavel"
assert classificar(40) == "muito quente"

print("Conversor OK.")
```

Todos passam. E é aqui que está a lição: **os testes que passam também ensinam**. Olhe os três casos escolhidos para a conversão — zero, cem e quarenta negativos. Não foram sorteados. São os três valores em que um erro de sinal ou de ordem das contas apareceria na hora. Se você tivesse testado só `celsius_para_fahrenheit(20) == 68`, um monte de implementação errada passaria.

OS VALORES DE BORDA

Erro quase nunca mora no meio da faixa. Mora na beirada: no zero, no vazio, no negativo, no primeiro e no último, no exatamente-igual-ao-limite. Ao escolher o que testar, comece pelas bordas — `classificar(15)` vale mais que `classificar(18)`, porque 15 é onde o `<` vira `<=` se alguém digitar errado.

Exercício 1: as primeiras afirmações do jogo

ARQUIVO: TESTES.PY · CRIE AGORA

Crie o arquivo `testes.py` **dentro da mesma pasta** dos outros quatro. Isso não é detalhe: é o que faz o `import` funcionar sem configuração nenhuma.

Comece com:

```
from personagem import Personagem
```

Faça:

1. Escreva um `assert` para cada uma das cinco regras abaixo, cada um com a sua mensagem.
2. Crie um `Personagem` novo para cada teste. Use `Personagem("Herói", 100, 20, 5, 0)` — cem de vida, vinte de ataque, cinco de defesa, zero poções.
3. Termine o arquivo com um `print` avisando que passou tudo.
4. Rode com `python testes.py`.

REGRA

COMO PROVOCAR

Curar não passa da vida máxima	Tome 15 de dano, cure 1000, e a vida tem que estar igual à vida máxima.
Curar negativo não faz nada	Guarde a vida antes, cure -30, compare.
O dano desconta a defesa	Receba 30 de dano com 5 de defesa. Faça a conta você mesmo e escreva o número.
Dano menor que a defesa não tira vida	Receba 3 de dano com 5 de defesa.
A vida nunca fica negativa	Receba 9999 de dano. A vida tem que ser exatamente 0, e <code>esta_vivo()</code> tem que ser falso.

CONFIRA RODANDO

Rode `python testes.py`. A saída inteira tem que ser a sua linha final, e mais nada.

Agora **faça um falhar de propósito**: troque um `==` por um valor errado e rode de novo. Você tem que ver o `AssertionError` com a sua mensagem e o número da linha. Guarde essa imagem — é o que você vai ver toda vez que quebrar alguma coisa daqui para a frente. Depois desfaça.

Um teste que você nunca viu falhar não é um teste: é uma linha de código que você acha que faz alguma coisa.

LEIA A FONTE

Procure na documentação do Python, em português, a página sobre a opção `-0` da linha de comando. Rode `python -0 testes.py` com um teste quebrado de propósito.

Responda: _____ o que aconteceu, e por que isso significa que `assert` nunca deve ser usado para validar dados de usuário no código do jogo?

Terminou o capítulo 1? O extra `E1.1` do anexo B depende dele.

Um arquivo só de testes

PROBLEMA → EXPLICAÇÃO → EXEMPLO → EXEMPLO → EXERCÍCIO

O problema

Você acabou de escrever cinco afirmações. Imagine trinta — que é quanto o seu jogo merece. Aí quebre alguma coisa no `personagem.py` e rode:

```
AssertionError: curar nao pode passar da vida maxima
```

Uma linha. O programa parou na terceira afirmação e as outras vinte e sete **não rodaram**. Você não sabe se elas passariam ou não. Conserta, roda de novo, para na sétima. Conserta, roda de novo, para na décima segunda.

São três problemas de uma vez, e o terceiro é o pior:

- Um erro esconde todos os outros.
- Não existe placar. Você nunca sabe quantos testes tem nem quantos passaram.
- Os testes compartilham o arquivo inteiro. Uma variável que um teste deixou suja é herdada pelo próximo — e o teste passa por um motivo que não é o certo, ou falha por um motivo que não é o dele.

A explicação: `unittest`

Isso é problema tão antigo e tão universal que o Python já vem com a solução instalada. O módulo se chama `unittest` e você não precisa baixar nada.

A ideia central é simples: em vez de afirmações soltas no meio do arquivo, **cada teste vira um método**. O `unittest` roda cada método separado dos outros, num ambiente limpo, e no fim dá o placar.

```
import unittest

class TesteCofrinho(unittest.TestCase):

    def test_depositar_soma_no_saldo(self):
        cofre = Cofrinho()
        cofre.depositar(50)
        self.assertEqual(cofre.saldo, 50)

    def test_deposito_negativo_e_ignorado(self):
        cofre = Cofrinho()
        cofre.depositar(-20)
        self.assertEqual(cofre.saldo, 0)

if __name__ == "__main__":
    unittest.main()
```

Quatro coisas novas nesse código, e cada uma tem uma razão de ser:

O QUÊ	POR QUÊ
Herda de <code>unittest.TestCase</code>	É de onde vêm os <code>assertAlgunaCoisa</code> . Herança, exatamente como você já usa desde a parte 1.
Métodos começam com <code>test_</code>	É assim que o <code>unittest</code> descobre o que é teste e o que é método auxiliar. Não é estilo: é como ele funciona.
<code>self.assertEqual(a, b)</code>	Faz o mesmo que <code>assert a == b</code> , mas com uma diferença enorme na hora do erro.
<code>unittest.main()</code>	Procura todas as classes de teste do arquivo, roda todos os métodos <code>test_</code> , e imprime o placar.

POR QUE `ASERTEQUAL` E NÃO `ASSERT ==`

Essa é a pergunta que todo mundo faz, e a resposta só aparece quando o teste falha. Compare as duas saídas para o mesmo erro:

```
AssertionError: o saldo deveria ser 50
```

```
AssertionError: 30 != 50
```

A primeira te diz o que era esperado. A segunda te diz o que era esperado **e o que aconteceu de verdade**. Aquele `30` é meio caminho do conserto: você já sabe que o saldo mexeu, só que menos do que devia.

É por isso que o `unittest` tem uma família inteira desses métodos, cada um sabendo imprimir bem o seu tipo de falha:

MÉTODO	USE QUANDO
<code>assertEqual(a, b)</code>	Os dois têm que ser iguais. É o que você mais vai usar.
<code>assertTrue(x)</code>	Algo tem que ser verdadeiro. <code>assertTrue(heroi.esta_vivo())</code> .
<code>assertFalse(x)</code>	Algo tem que ser falso.
<code>assertIsNone(x)</code>	O método devolveu <code>None</code> — o caso do <code>Inventario.tirar</code> com número inválido.
<code>assertIs(a, b)</code>	São o mesmo objeto , não só objetos iguais. Serve para provar que a mochila devolveu a poção que você pôs, e não outra.
<code>assertGreater(a, b)</code>	<code>a > b</code> . Tem também <code>assertLess</code> , <code>assertGreaterEqual</code> e <code>assertLessEqual</code> .
<code>assertIn(x, colecao)</code>	<code>x</code> está dentro de uma lista, de um dicionário ou de um texto.

O `SETUP`: O PREPARO QUE SE REPETE

Repare que os dois testes do cofrinho começam com a mesma linha, `cofre = Cofrinho()`. Com dez testes seriam dez linhas iguais — e ontem você aprendeu o nome disso.

O `unittest` tem um lugar para essa linha. Um método chamado `setUp`, que ele executa **antes de cada teste**, automaticamente:

```
class TesteCofrinho(unittest.TestCase):

    def setUp(self):
        self.cofre = Cofrinho()

    def test_depositar_soma_no_saldo(self):
        self.cofre.depositar(50)
        self.assertEqual(self.cofre.saldo, 50)

    def test_deposito_negativo_e_ignorado(self):
        self.cofre.depositar(-20)
        self.assertEqual(self.cofre.saldo, 0)
```

ANTES DE CADA TESTE, NÃO UMA VEZ

O `setUp` roda de novo para cada método. Não é um `__init__`. São dez cofrinhos diferentes para dez testes, e é justamente isso que impede um teste de sujar o outro — o terceiro problema da lista lá do começo do capítulo, resolvido sem você fazer nada.

E repare que ele é `setUp`, com U maiúsculo. Esse nome não é escolha sua: o `unittest` procura por ele com essa grafia exata. `setup` minúsculo é um método comum que nunca vai ser chamado, e o seu teste vai falhar com `AttributeError` num objeto que nunca foi criado.

Exemplo 1: o placar do vôlei

Vôlei tem uma regra que serve de presente para quem está aprendendo a testar: ganha quem chegar a 25 pontos, **mas** precisa de dois de vantagem. Se empatar em 24, o set continua até alguém abrir dois.

```
class Set:
    def __init__(self):
        self.casa = 0
        self.visitante = 0

    def ponto_casa(self):
        self.casa += 1

    def ponto_visitante(self):
        self.visitante += 1

    def terminou(self):
        if self.casa < 25 and self.visitante < 25:
            return False
        return abs(self.casa - self.visitante) >= 2

    def vencedor(self):
        if not self.terminou():
            return None
        if self.casa > self.visitante:
            return "casa"
        return "visitante"
```

```

import unittest

class TesteSet(unittest.TestCase):

    def setUp(self):
        self.set = Set()

    def marcar(self, casa, visitante):
        """Auxiliar. Nao comeca com test_, entao nao e um teste."""
        for _ in range(casa):
            self.set.ponto_casa()
        for _ in range(visitante):
            self.set.ponto_visitante()

    def test_set_novo_nao_terminou(self):
        self.assertFalse(self.set.terminou())
        self.assertIsNone(self.set.vencedor())

    def test_25_a_20_termina(self):
        self.marcar(25, 20)
        self.assertTrue(self.set.terminou())
        self.assertEqual(self.set.vencedor(), "casa")

    def test_25_a_24_nao_termina(self):
        self.marcar(25, 24)
        self.assertFalse(self.set.terminou())

    def test_26_a_24_termina(self):
        self.marcar(26, 24)
        self.assertTrue(self.set.terminou())
        self.assertEqual(self.set.vencedor(), "casa")

    def test_30_a_28_termina_com_o_visitante(self):
        self.marcar(28, 30)
        self.assertTrue(self.set.terminou())
        self.assertEqual(self.set.vencedor(), "visitante")

if __name__ == "__main__":
    unittest.main()

```

A SAÍDA

```

.....
-----
Ran 5 tests in 0.001s

OK

```

Aqueles cinco pontinhos são os cinco testes que passaram — um ponto por teste, na ordem em que rodaram. Agora o mais importante deste exemplo: repare no método `marcar`. Ele está dentro da classe de teste, usa `self`, e **não é um teste**, porque não começa com `test_`. É um ajudante. Sem ele, `test_30_a_28` teria cinquenta e oito linhas.

E repare em quais testes foram escritos: 25×20, 25×24, 26×24. São as bordas do capítulo 1 outra vez. 25×20 prova que a regra normal funciona; 25×24 prova que os dois de vantagem são exigidos; 26×24 prova que a exceção termina quando deve. Um teste de 25×3 não acrescenta nada que o de 25×20 já não diga.

Exemplo 2: a máquina de salgadinhos

Este exemplo mostra a saída quando alguma coisa dá errado, que é o que você mais vai ver na vida real.

```
class Maquina:
    def __init__(self):
        self.estoque = {"salgadinho": 3, "chocolate": 1}
        self.caixa = 0

    def vender(self, produto, dinheiro):
        if produto not in self.estoque:
            return "produto inexistente"
        if self.estoque[produto] == 0:
            return "esgotado"
        if dinheiro < 5:
            return "dinheiro insuficiente"
        self.estoque[produto] -= 1
        self.caixa += 5
        return "troco: " + str(dinheiro - 5)
```

```
class TesteMaquina(unittest.TestCase):

    def setUp(self):
        self.maquina = Maquina()

    def test_venda_normal_da_troco(self):
        self.assertEqual(self.maquina.vender("salgadinho", 10), "troco: 5")

    def test_venda_normal_baixa_o_estoque(self):
        self.maquina.vender("salgadinho", 5)
        self.assertEqual(self.maquina.estoque["salgadinho"], 2)

    def test_venda_normal_entra_no_caixa(self):
        self.maquina.vender("salgadinho", 20)
        self.assertEqual(self.maquina.caixa, 5)

    def test_produto_que_nao_existe(self):
        self.assertEqual(self.maquina.vender("refrigerante", 10),
                         "produto inexistente")

    def test_esgotado_depois_da_ultima_unidade(self):
        self.maquina.vender("chocolate", 5)
        self.assertEqual(self.maquina.vender("chocolate", 5), "esgotado")

    def test_dinheiro_de_menos_nao_baixa_estoque(self):
        self.maquina.vender("salgadinho", 2)
        self.assertEqual(self.maquina.estoque["salgadinho"], 3)
        self.assertEqual(self.maquina.caixa, 0)
```

Suponha agora que alguém mexa no `vender` e troque `self.caixa += 5` por `self.caixa += dinheiro`. Parece inofensivo. A saída:

```

..F...
=====
FAIL: test_venda_normal_entra_no_caixa (__main__.TesteMaquina)
-----
Traceback (most recent call last):
  File "testes.py", line 18, in test_venda_normal_entra_no_caixa
    self.assertEqual(self.maquina.caixa, 5)
AssertionError: 20 != 5

-----
Ran 6 tests in 0.002s

FAILED (failures=1)

```

Leia essa saída com calma, porque ela é o seu dia a dia de agora em diante:

- `..F...` — o terceiro teste falhou. Os outros cinco rodaram assim mesmo. É exatamente o que os `assert` soltos não davam.
- O nome do método falho aparece inteiro. Se você nomeou bem, o nome já diz qual regra quebrou, antes de você ler mais nada.
- `20 != 5` — o caixa ficou com o valor que o cliente deu, em vez do preço. Isso praticamente aponta a linha errada com o dedo.
- `FAILED (failures=1)` — o placar final. Um erro entre seis.

Falha e erro não são a mesma coisa. `FAIL` quer dizer que o teste rodou até o fim e a afirmação era falsa — o código está errado. `ERROR` quer dizer que o teste explodiu no meio, com `TypeError`, `AttributeError` ou parecido — e aí pode ser o código ou pode ser o teste mal escrito. O `unittest` conta os dois separados de propósito.

Exercício 2: passar tudo para `unittest`

ARQUIVO: TESTES.PY

Faça:

1. No topo do arquivo, acrescente `import unittest` e `from itens import Inventario, PocaDeVida`.
2. Crie a classe `TesteVida(unittest.TestCase)`. Dê a ela um `setUp` que crie `self.heroi = Personagem("Herói", 100, 20, 5, 0)`.
3. Transforme cada um dos cinco `assert` do capítulo 1 num método `test_` dessa classe, usando `assertEqual`, `assertTrue` e `assertFalse` no lugar do `assert` cru. Apague os `assert` soltos e o `print` do fim.
4. Acrescente um sexto teste nessa classe: defender antes de receber o dano faz o dano cair pela metade. Faça a conta você mesmo: 30 de dano, 5 de defesa, defendendo. Escreva o número que você calculou e veja se o código concorda.
5. Crie a classe `TesteInventario`, com `setUp` criando uma mochila vazia, e quatro testes:

TESTE

O QUE AFIRMAR

Mochila nova está vazia	<code>esta_vazio()</code> verdadeiro e <code>quantidade()</code> igual a zero.
Adicionar aumenta a quantidade	Duas poções, quantidade dois.
Tirar devolve o item certo	Guarde a poção numa variável antes de adicionar, e use <code>assertIs</code> para provar que voltou a mesma . Depois a mochila tem que estar vazia.
Tirar número inválido	<code>tirar(7)</code> e <code>tirar(0)</code> devolvem <code>None</code> , e a mochila continua com o que tinha.

6. No fim do arquivo, o bloco `if __name__ == "__main__": unittest.main()`.

CONFIRA RODANDO

Rode `python testes.py`. Você tem que ver dez pontinhos, `Ran 10 tests` e `OK`.

Agora quebre um de propósito e leia a saída inteira: os outros nove continuaram rodando, e a falha te disse o nome do método, o valor esperado e o valor que veio. Desfaça.

Por último, renomeie um método de `test_curar...` para `curar...` e rode. Ele some do placar sem nenhum aviso — `Ran 9 tests`, e nada dizendo que sumiu um. Esse silêncio é a armadilha do capítulo: um teste com nome errado não falha, ele simplesmente não existe. Desfaça.

LEIA A FONTE

Procure a documentação do módulo `unittest` na página oficial do Python em português, e ache a lista completa dos métodos de asserção.

Responda: _____ qual é a diferença entre `assertEqual` e `assertIs`, e por que o teste da mochila usa o segundo?

Terminou o capítulo 2? Os extras `E2.1` e `E2.2` dependem dele.

Testar o que sorteia

PROBLEMA → EXPLICAÇÃO → EXEMPLO → EXEMPLO → EXERCÍCIO

O problema

Até agora você testou coisas mansas: curar, receber dano, guardar item. Todas dão o mesmo resultado toda vez. Agora tente testar isto:

```
def calcular_dano(self):
    return random.randint(self.ataque - VARIACAO_DO_DANO,
                          self.ataque + VARIACAO_DO_DANO)
```

Escreva a afirmação óbvia e veja o que acontece:

```
def test_o_dano(self):
    heroi = Personagem("Herói", 100, 20, 5, 0)
    self.assertEqual(heroi.calcular_dano(), 20) # ???
```

Esse teste passa uma vez a cada sete execuções. Nas outras seis ele falha, sem que exista nada de errado no código. É o pior tipo de teste que existe, e ele tem nome próprio na profissão: um teste **instável** — em inglês, *flaky*.

POR QUE INSTÁVEL É PIOR QUE AUSENTE

Um teste que falha às vezes ensina a equipe inteira a ignorar o vermelho. Depois de três falhas falsas, a reação a uma falha vira "roda de novo". E aí, no dia em que a falha for de verdade, alguém vai rodar de novo, ver verde, e seguir em frente.

A explicação: três saídas, em ordem

Existem três formas de testar código que sorteia. Elas não são equivalentes — a primeira é quase sempre a certa, e as outras duas servem para casos específicos.

SAÍDA 1: AFIRMAR A PROPRIEDADE, NÃO O VALOR

A pergunta que destrava tudo é: **o que é verdade em toda rodada, e não só em uma?**

O valor exato do dano você não sabe. Mas você sabe que ele nunca é menor que `ataque - 3` nem maior que `ataque + 3`. **Isso** é a regra. O número sorteado é só uma consequência dela.

```
def test_o_dano_fica_sempre_na_faixa(self):
    heroi = Personagem("Herói", 100, 20, 5, 0)
    menor = heroi.ataque - VARIACAO_DO_DANO
    maior = heroi.ataque + VARIACAO_DO_DANO

    for _ in range(200):
        dano = heroi.calcular_dano()
        self.assertGreaterEqual(dano, menor)
        self.assertLessEqual(dano, maior)
```

Duzentas rodadas, duzentas afirmações, zero instabilidade — porque a coisa afirmada é verdadeira em toda rodada possível. E repare que o teste importa `VARIACAO_DO_DANO` em vez de escrever `3`. Se amanhã a variação virar 5, o teste continua correto sozinho. Escrever o `3` à mão faria o teste falhar numa mudança *legítima* do jogo, que é outra forma de ficar instável.

SAÍDA 2: FIXAR A SEMENTE

Sorteio de computador não é sorteio de verdade. O `random` é uma máquina que produz uma sequência fixa de números a partir de um valor inicial, chamado **semente**. Mesma semente, mesma sequência — sempre, em qualquer computador.

```
random.seed(14)
print(random.randint(1, 20)) # 4
print(random.randint(1, 20)) # 20
print(random.randint(1, 20)) # 17
```

Rode isso dez vezes. Sai a mesma coisa nas dez. Isso te dá poder de escolher a sorte: com a semente 14, a primeira rolagem é 4 — que é menos que `ROLAGEM_MINIMA_PARA_ACERTAR`, ou seja, um erro de ataque. Você acabou de conseguir testar o caminho do erro sem esperar dar sorte.

O PREÇO DA SEMENTE

O teste passa a depender da **ordem exata** das chamadas ao `random`. Se amanhã alguém acrescentar um sorteio qualquer antes — uma chance de acerto crítico, por exemplo — todos os números seguintes mudam e o seu teste quebra sem que nada esteja errado. Semente serve para forçar um caminho específico. Não serve como jeito padrão de testar sorteio.

SAÍDA 3: TIRAR O SORTEIO DE DENTRO

A terceira saída é mudar o desenho: em vez de o método sortear, ele **recebe** o número de fora. Aí o teste passa o número que quiser e não existe mais sorteio nenhum para domar.

```
def atacar(self, alvo, rolagem=None):
    if rolagem is None:
        rolagem = random.randint(1, LADOS_DO_DADO)
    ...
```

Isso se chama **injeção**, e é uma técnica poderosa e cara: você mudou a classe para poder testá-la. Às vezes isso melhora o desenho de verdade; às vezes só enche o código de parâmetros que ninguém usa jogando. Hoje você não vai fazer isso no jogo — fica no extra E3.1, para você formar opinião com as mãos.

SAÍDA	USE QUANDO
Propriedade	Quase sempre. É a primeira coisa a tentar.
Semente	Você precisa forçar um caminho específico — o erro, o crítico, o pior caso.
Injeção	O sorteio atrapalha tanto que vale mudar o desenho por causa dele.

Exemplo 1: o dado de tabuleiro

```
import random

class Dado:
    def __init__(self, lados):
        self.lados = lados

    def rolar(self):
        return random.randint(1, self.lados)
```

```
class TesteDado(unittest.TestCase):

    def test_o_valor_fica_sempre_na_faixa(self):
        dado = Dado(6)
        for _ in range(1000):
            valor = dado.rolar()
            self.assertGreaterEqual(valor, 1)
            self.assertLessEqual(valor, 6)

    def test_todos_os_lados_saem_alguma_hora(self):
        dado = Dado(6)
        vistos = set()
        for _ in range(1000):
            vistos.add(dado.rolar())
        self.assertEqual(vistos, {1, 2, 3, 4, 5, 6})

    def test_dado_de_vinte_chega_a_vinte(self):
        dado = Dado(20)
        vistos = set()
        for _ in range(2000):
            vistos.add(dado.rolar())
        self.assertIn(20, vistos)
        self.assertIn(1, vistos)
```

O segundo teste é o interessante. Ele pega uma classe de erro que o teste de faixa não pega: se alguém escrever `random.randint(1, self.lados - 1)`, a faixa continua válida — todo valor continua entre 1 e 6 — mas o 6 nunca sai. Só um teste que olha a **distribuição** percebe.

E repare que mesmo esse teste é escrito com cuidado para não ser instável. Mil rolagens de um dado de seis lados sem que um dos valores apareça é um evento tão improvável

que não acontece na prática. Se fossem dez rolagens, o teste falharia de vez em quando — e voltaríamos ao problema do começo do capítulo.

Exemplo 2: o sorteio da rifa

Aqui o sorteio é o assunto, não um detalhe, e a semente é a ferramenta certa.

```
import random

class Rifa:
    def __init__(self):
        self.numeros = {}

    def vender(self, numero, pessoa):
        if numero in self.numeros:
            return False
        self.numeros[numero] = pessoa
        return True

    def sortear(self):
        if not self.numeros:
            return None
        vendidos = sorted(self.numeros.keys())
        premiado = random.choice(vendidos)
        return self.numeros[premiado]
```

```
class TesteRifa(unittest.TestCase):

    def setUp(self):
        self.rifa = Rifa()

    def test_numero_ja_vendido_e_recusado(self):
        self.assertTrue(self.rifa.vender(7, "Ana"))
        self.assertFalse(self.rifa.vender(7, "Bruno"))
        self.assertEqual(self.rifa.numeros[7], "Ana")

    def test_rifa_vazia_nao_sorteia(self):
        self.assertIsNone(self.rifa.sortear())

    def test_com_um_numero_so_o_ganhador_e_certo(self):
        self.rifa.vender(3, "Ana")
        self.assertEqual(self.rifa.sortear(), "Ana")

    def test_o_ganhador_sempre_comprou(self):
        self.rifa.vender(1, "Ana")
        self.rifa.vender(2, "Bruno")
        self.rifa.vender(3, "Carla")
        for _ in range(100):
            self.assertIn(self.rifa.sortear(), ["Ana", "Bruno", "Carla"])

    def test_com_a_semente_5_o_resultado_e_sempre_o_mesmo(self):
        self.rifa.vender(1, "Ana")
        self.rifa.vender(2, "Bruno")
        random.seed(5)
        primeiro = self.rifa.sortear()
        random.seed(5)
        segundo = self.rifa.sortear()
        self.assertEqual(primeiro, segundo)
```

Cinco testes e nenhum deles depende de sorte. Repare no terceiro: com um comprador só, o sorteio deixa de ser sorteio — o resultado é certo. Reduzir o caso até a aleatoriedade sumir é um truque que resolve muito mais teste do que parece.

E olhe o `sorted` dentro do `sortear`. Sem ele, a ordem em que os números entram no dicionário influencia o que o `random.choice` escolhe, e o teste da semente vira loteria.

Fazer o código ser previsível também é parte de fazê-lo testável.

Exercício 3: domar o dano

ARQUIVO: TESTES.PY

Acrescente ao topo do arquivo:

```
import random

from personagem import Personagem, VARIACAO_DO_DANO
from classes import Guerreiro
```

Crie a classe `TesteSorteio` com três testes:

1. **A faixa do dano.** Duzentas chamadas a `calcular_dano`, todas entre `ataque - VARIACAO_DO_DANO` e `ataque + VARIACAO_DO_DANO`. Importe a constante; não escreva o `3`.
2. **O Guerreiro nunca erra.** Crie um Guerreiro e um boneco com `Personagem("Boneco", 10000, 1, 0, 0)` — muita vida e zero defesa, para que qualquer dano apareça. Ataque cinquenta vezes e afirme, a cada uma, que a vida do boneco baixou em relação ao turno anterior. Este teste não precisa de semente nenhuma: se ele passar cinquenta vezes seguidas, é porque o Guerreiro realmente não tem sorteio de erro.
3. **Os outros erram.** Use `random.seed(14)`, crie um `Personagem` comum e um boneco, e ataque **uma** vez. Com essa semente a primeira rolagem é 4, abaixo de `ROLAGEM_MINIMA_PARA_ACERTAR`. Afirme que a vida do boneco **não** mudou.

POR QUE O BONECO TEM DEFESA ZERO

Se o alvo tivesse defesa, um dano baixo poderia ser absorvido e a vida não baixaria — e o seu teste falharia por causa da defesa, não do ataque. Um bom teste elimina tudo que não é o assunto dele. Vida alta pelo mesmo motivo: com pouca vida o boneco morre no meio do laço e as últimas afirmações testam outra coisa.

CONFIRA RODANDO

Rode dez vezes seguidas. As dez têm que dar `OK`. Se uma falhar, você escreveu um teste instável — releia qual afirmação depende de sorte.

Agora o teste que prova o teste: no `personagem.py`, troque `VARIACAO_DO_DANO` para `3000` e rode. O teste da faixa tem que continuar passando (a faixa mudou junto), mas o do Guerreiro pode falhar de um jeito interessante — dano negativo não tira vida. Volte para 3 e pense no que isso te contou sobre o que cada teste está de fato afirmando.

Terminou o capítulo 3? O extra **E3.1** depende dele.

Testar o que imprime

PROBLEMA → EXPLICAÇÃO → EXEMPLO → EXEMPLO → EXERCÍCIO

O problema

Rode os seus testes agora e olhe a tela com atenção. Entre os pontinhos aparece isto:

```
..Herói preparou a defesa
..Os ossos rangem
..Teste atacou sem falhar e causou 19 de dano
...
```

O jogo está falando no meio do relatório de testes. E não é só feio: é o sintoma de um problema de desenho que atravessa o jogo inteiro.

Olhe o `atacar`:

```
def atacar(self, alvo):
    rolagem = random.randint(1, LADOS_DO_DADO)

    if rolagem < ROLAGEM_MINIMA_PARA_ACERTAR:
        print(self.nome, "errou o ataque")
        return

    dano = self.calcular_dano()
    alvo.receber_dano(dano)
    print(self.nome, "causou", dano, "de dano")
```

Esse método não devolve nada. Ele decide, aplica e conta na tela — e o resultado do trabalho dele sai por `print`, que é um lugar de onde não se pega valor de volta. Você não tem como escrever `self.assertEqual(heroi.atacar(alvo), ...)`, porque a resposta é sempre `None`.

E o `turno_do_jogador` é pior: ele chama `input`. Se um teste chamar esse método, o teste **trava para sempre**, esperando alguém digitar num terminal que ninguém está olhando.

A explicação: separar decidir de mostrar

Você já viu esse movimento duas vezes neste curso, e agora ele volta pela terceira, com um motivo novo.

QUANDO	O QUE SE SEPAROU
Parte 9, o <code>agir</code>	Decidir uma ação de executar a ação.
Parte 10, capítulo 1	O <code>media()</code> devolvia valor; quem chamou é que imprimia.
Hoje	Calcular o resultado de mostrar o resultado.

A regra geral: **um método que devolve valor é testável; um método que só imprime, não.** Isso não quer dizer que `print` é proibido — o jogo precisa falar. Quer dizer que o `print` tem que ficar na beirada, e a decisão tem que acontecer em algum lugar que devolve.

O OUTRO CAMINHO: TESTAR O EFEITO

Antes de sair mudando o jogo, existe uma saída que não exige mudar nada. O `atacar` não devolve o dano, mas ele **deixa rastro**: a vida do alvo mudou. E vida você consegue ler.

```
def test_atacar_tira_vida_do_alvo(self):
    goblin = Goblin()
    alvo = Personagem("Boneco", 500, 1, 0, 0)

    for _ in range(30):
        goblin.atacar(alvo)

    self.assertLess(alvo.vida, 500)
```

Em vez de afirmar sobre o que o método disse, você afirma sobre o **estado do mundo depois** que ele rodou. Isso funciona sem tocar numa linha do jogo, e vale para quase tudo: a habilidade entrou em recarga, a mochila esvaziou, o bônus está ativo, o inimigo morreu.

A ORDEM PARA TENTAR

Primeiro tente afirmar sobre o estado que sobrou. **Só se não der**, mude o código para devolver valor. Refatorar para testar é legítimo, mas é a segunda opção, não a primeira — senão o dia acaba com você tendo reescrito o jogo inteiro para testar duas coisas.

Exemplo 1: a agenda de consultas

ANTES: DECIDE E IMPRIME NA MESMA LINHA

```

class Agenda:
    def __init__(self):
        self.horarios = {}

    def marcar(self, hora, paciente):
        if hora in self.horarios:
            print("Horario", hora, "ja ocupado por", self.horarios[hora])
            return
        if hora < 8 or hora > 17:
            print("Fora do horario de atendimento")
            return
        self.horarios[hora] = paciente
        print("Marcado:", paciente, "as", hora)

```

Para testar a regra "não marca fora do expediente", você precisaria capturar o texto impresso. Dá para fazer, mas é desconfortável: o teste passaria a depender da **redação da frase**, e trocar "Fora do horario de atendimento" por "Estamos fechados" quebraria um teste sem que regra nenhuma tivesse mudado.

DEPOIS: DECIDE E DEVOLVE; QUEM CHAMOU IMPRIME

```

class Agenda:
    def __init__(self):
        self.horarios = {}

    def marcar(self, hora, paciente):
        """Devolve None se deu certo, ou o motivo da recusa."""
        if hora in self.horarios:
            return "ocupado"
        if hora < 8 or hora > 17:
            return "fora do expediente"
        self.horarios[hora] = paciente
        return None

def marcar_na_tela(agenda, hora, paciente):
    motivo = agenda.marcar(hora, paciente)
    if motivo == "ocupado":
        print("Horario", hora, "ja ocupado por", agenda.horarios[hora])
    elif motivo == "fora do expediente":
        print("Fora do horario de atendimento")
    else:
        print("Marcado:", paciente, "as", hora)

```

```

class TesteAgenda(unittest.TestCase):

    def setUp(self):
        self.agenda = Agenda()

    def test_marcar_num_horario_livre_da_certo(self):
        self.assertIsNone(self.agenda.marcar(10, "Ana"))
        self.assertEqual(self.agenda.horarios[10], "Ana")

    def test_horario_ocupado_e_recusado(self):
        self.agenda.marcar(10, "Ana")
        self.assertEqual(self.agenda.marcar(10, "Bruno"), "ocupado")
        self.assertEqual(self.agenda.horarios[10], "Ana")

    def test_antes_das_oito(self):
        self.assertEqual(self.agenda.marcar(7, "Ana"), "fora do expediente")

    def test_as_oito_em_ponto_pode(self):
        self.assertIsNone(self.agenda.marcar(8, "Ana"))

    def test_as_dezessete_pode(self):
        self.assertIsNone(self.agenda.marcar(17, "Ana"))

    def test_as_dezoito_nao_pode(self):
        self.assertEqual(self.agenda.marcar(18, "Ana"), "fora do expediente")

```

Repare que a saída na tela ficou **igualzinha** — as mesmas frases, nas mesmas situações. O que mudou é onde elas moram. E veja os quatro últimos testes: 7, 8, 17 e 18. As bordas, de novo. Se alguém trocar `hora > 17` por `hora >= 17`, exatamente um teste fica vermelho e ele diz qual hora quebrou.

Exemplo 2: a validação de senha

Um caso em que a coisa devolvida não é um texto, mas uma lista — e o teste fica melhor por causa disso.

ANTES

```

def validar(senha):
    if len(senha) < 8:
        print("A senha precisa de pelo menos 8 caracteres")
    if senha.lower() == senha:
        print("A senha precisa de uma letra maiuscula")
    if not any(c.isdigit() for c in senha):
        print("A senha precisa de um numero")

```

DEPOIS

```
def problemas_da_senha(senha):
    problemas = []
    if len(senha) < 8:
        problemas.append("curta")
    if senha.lower() == senha:
        problemas.append("sem maiuscula")
    if not any(c.isdigit() for c in senha):
        problemas.append("sem numero")
    return problemas

def validar(senha):
    mensagens = {
        "curta": "A senha precisa de pelo menos 8 caracteres",
        "sem maiuscula": "A senha precisa de uma letra maiuscula",
        "sem numero": "A senha precisa de um numero",
    }
    for problema in problemas_da_senha(senha):
        print(mensagens[problema])
```

```
class TesteSenha(unittest.TestCase):

    def test_senha_boa_nao_tem_problema(self):
        self.assertEqual(problemas_da_senha("Girassol9"), [])

    def test_senha_curta(self):
        self.assertIn("curta", problemas_da_senha("Ab3"))

    def test_senha_sem_maiuscula(self):
        self.assertIn("sem maiuscula", problemas_da_senha("girassol9"))

    def test_senha_sem_numero(self):
        self.assertIn("sem numero", problemas_da_senha("Girassol1"))

    def test_senha_pessima_acumula_os_tres(self):
        self.assertEqual(len(problemas_da_senha("abc")), 3)

    def test_exatamente_oito_caracteres_serve(self):
        self.assertNotIn("curta", problemas_da_senha("Girassol9"))
```

Duas coisas aconteceram sozinhas quando a função passou a devolver. Primeiro, o dicionário de mensagens apareceu — é o capítulo 3 de ontem, condicional virando tabela, sem ninguém ter planejado. Segundo, ficou trivial testar que os problemas **acumulam**: uma senha ruim de três jeitos devolve três problemas. Com `print`, testar isso exigiria contar linhas de texto.

E se eu quiser mesmo testar o texto impresso? Dá. O Python tem `contextlib.redirect_stdout`, que desvia o `print` para dentro de uma variável. É o extra [E4.1](#). Mas repare no que você estaria testando: a redação de uma frase. Isso trava o texto do jogo — trocar uma palavra passa a quebrar teste. Às vezes é exatamente o que se quer; na maioria das vezes, não.

Exercício 4: testar pelo efeito

ARQUIVO: `TESTES.PY`

Nenhuma linha do jogo muda neste exercício. Tudo aqui se testa pelo estado que sobra.

Crie a classe `TesteEfeito` com quatro testes:

1. **Atacar tira vida.** Um Goblin, um boneco com 500 de vida e defesa 0, trinta ataques. No fim, a vida tem que ser menor que 500. Trinta ataques porque um só pode errar — assim a chance de errar as trinta é praticamente nula, e o teste não fica instável.
2. **A poção cura quem bebe.** Tome 55 de dano, guarde a vida, use uma `PocaoDeVida` com `usar(heroi, None)`, e afirme que a vida subiu exatamente 25. O `None` no lugar do inimigo é de propósito: a poção não usa o inimigo para nada, e passar `None` prova isso — se um dia alguém fizer a poção mexer no inimigo, este teste quebra com `AttributeError` e você fica sabendo.
3. **A bomba machuca o inimigo e não o dono.** Use uma `Bomba` com dono e inimigo de verdade. A vida do inimigo baixa, a do dono não.
4. **O Goblin se defende com pouca vida.** Deixe o Goblin quase morto (`receber_dano(1000)` e depois `curar(5)`), mande ele agir contra um boneco, e afirme que `goblin.defendendo` ficou verdadeiro. Este é o primeiro teste que você escreve sobre uma **decisão** de inimigo, e ele não olha a tela: olha o estado.

POR QUE NÃO TESTAMOS O MAIN

O `escolher_personagem`, o `turno_do_jogador` e o `abrir_mochila` chamam `input`. Teste que chama `input` trava. Isso não é acidente do seu código — é o motivo pelo qual programas sérios empurram entrada e saída para as beiradas: o miolo, que é onde moram as regras, tem que ser alcançável sem teclado. O seu miolo é o `personagem.py` e o `itens.py`, e é por isso que quase tudo que você testou hoje mora lá.

CONFIRA RODANDO

Ran 17 tests e OK, mais o barulho do jogo no meio dos pontinhos. O barulho é esperado — se ele te incomodar, o extra E4.1 ensina a calar.

Prova do teste 2: no `itens.py`, mude a cura da poção de 25 para 30 e rode. Um teste tem que falhar, e a mensagem tem que ser `70 != 75` ou parecida — dizendo o valor esperado e o obtido. Volte para 25.

Terminou o capítulo 4? O extra E4.1 depende dele.

O teste que pega a regra

PROBLEMA → EXPLICAÇÃO → EXEMPLO → EXEMPLO → EXERCÍCIO

O problema

Volte ao anexo A de ontem e releia os erros 5, 6 e 7. Os três começam com a mesma palavra:

ERRO DE ONTEM	O QUE ACONTECIA
5	Sem erro: a habilidade funcionava todo turno, sem recarga.
6	Sem erro: dois elixires seguidos e o ataque subia duas vezes.
7	Sem erro: o bônus subia e ficava alto para sempre.

Nenhum dos três aparece na tela. Nenhum dos três quebra o jogo. Os três fazem o jogo continuar rodando **com a regra errada** — e você só descobre se souber que a regra existe, chegar na situação exata e reparar no número.

Repare no que os três têm em comum: as regras já existiam e funcionavam de manhã. O que quase aconteceu ontem foi **perdê-las durante uma mudança** que não tinha nada a ver com elas.

O INCÔMODO

Uma regra que só existe na cabeça de quem escreveu vai embora na primeira refatoração. Uma regra escrita num teste sobrevive a você.

A explicação: teste de regressão

Regressão é quando alguma coisa que funcionava para de funcionar. Um teste de regressão é um teste escrito especificamente para que um bug conhecido não volte.

Ele nasce de um jeito muito característico, e é assim na indústria inteira:

1. Alguém encontra um bug.
2. **Antes de consertar**, escreve-se um teste que reproduz o bug. Esse teste falha — é para falhar. Ele acabou de provar que o bug existe e que o teste sabe enxergá-lo.
3. Conserta-se o código.
4. O teste passa. Agora ele fica no arquivo para sempre, montando guarda.

O passo 2 é o que quase todo mundo pula, e é o único que importa de verdade. Um teste escrito depois do conserto nunca foi visto falhando — então você não tem prova nenhuma

de que ele detectaria o problema.

VERMELHO ANTES DE VERDE

Todo teste que você escrever hoje, faça falhar de propósito uma vez. Quebre a regra no código do jogo, veja o vermelho, desfaça, veja o verde. É a única forma de saber que o teste está de fato ligado ao código que ele diz testar.

TESTAR A REGRA, NÃO A IMPLEMENTAÇÃO

Existe uma armadilha aqui, e ela custa caro:

```
def test_o_elixir(self):
    heroi = Personagem("Herói", 100, 20, 5, 0)
    ElixirDeFuria().usar(heroi, None)
    self.assertEqual(heroi.bonus_ataque.turnos, 3)
    self.assertEqual(heroi.bonus_ataque.quantidade, 6)
```

```
def test_o_elixir_aumenta_o_ataque(self):
    heroi = Personagem("Herói", 100, 20, 5, 0)
    ataque_original = heroi.ataque
    ElixirDeFuria().usar(heroi, None)
    self.assertEqual(heroi.ataque, ataque_original + 6)
```

O primeiro testa **como** o código foi escrito por dentro: que existe um objeto chamado `bonus_ataque` com um atributo `turnos`. Se amanhã você reorganizar isso — exatamente o que você fez ontem! — o teste quebra sem que o jogo tenha mudado em nada.

O segundo testa **o que o jogador sente**: o ataque subiu 6. Essa frase continua verdadeira independentemente de como o bônus é guardado por dentro.

É a mesma lição do encapsulamento, chegando por outra porta: quem olha por dentro fica preso ao lado de dentro.

Exemplo 1: o estoque da padaria

Um bug que só aparece na segunda chamada — o tipo mais comum de regressão.

```
class Padaria:
    def __init__(self):
        self.paes = 100
        self.vendidos = 0

    def vender(self, quantidade):
        if quantidade > self.paes:
            return False
        self.vendidos = quantidade      # BUG: sobrescreve em vez de somar
        self.paes -= quantidade
        return True
```

Um teste ingênuo não pega:

```
def test_vender_conta_certo(self):
    padaria = Padaria()
    padaria.vender(10)
    self.assertEqual(padaria.vendidos, 10)    # passa!
```

O teste que pega:

```
def test_duas_vendas_somam(self):
    padaria = Padaria()
    padaria.vender(10)
    padaria.vender(5)
    self.assertEqual(padaria.vendidos, 15)    # falha: 5 != 15
    self.assertEqual(padaria.paes, 85)
```

DUAS VEZES

Sempre que um método acumula alguma coisa, chame-o **duas vezes** no teste. Uma chamada só não distingue somar de sobrescrever. Guarde essa: metade dos bugs de estado se esconde na segunda chamada.

Exemplo 2: o cupom da lanchonete

```
class Pedido:
    def __init__(self):
        self.itens = []
        self.desconto = 0
        self.cupons_usados = []

    def adicionar(self, nome, preco):
        self.itens.append((nome, preco))

    def aplicar_cupom(self, codigo):
        if codigo in self.cupons_usados:
            return False
        self.cupons_usados.append(codigo)
        self.desconto += 10
        return True

    def total(self):
        soma = 0
        for nome, preco in self.itens:
            soma += preco
        return max(0, soma - self.desconto)
```

```

class TestePedido(unittest.TestCase):

    def setUp(self):
        self.pedido = Pedido()
        self.pedido.adicionar("X-salada", 22)
        self.pedido.adicionar("Suco", 8)

    def test_total_sem_cupom(self):
        self.assertEqual(self.pedido.total(), 30)

    def test_cupom_desconta_dez(self):
        self.pedido.aplicar_cupom("PROMO")
        self.assertEqual(self.pedido.total(), 20)

    def test_o_mesmo_cupom_nao_vale_duas_vezes(self):
        self.assertTrue(self.pedido.aplicar_cupom("PROMO"))
        self.assertFalse(self.pedido.aplicar_cupom("PROMO"))
        self.assertEqual(self.pedido.total(), 20)

    def test_dois_cupons_diferentes_acumulam(self):
        self.pedido.aplicar_cupom("PROMO")
        self.pedido.aplicar_cupom("NOVO")
        self.assertEqual(self.pedido.total(), 10)

    def test_o_total_nunca_fica_negativo(self):
        for codigo in ["A", "B", "C", "D", "E"]:
            self.pedido.aplicar_cupom(codigo)
            self.assertEqual(self.pedido.total(), 0)

```

Os dois últimos testes são os que valem. O quarto documenta uma decisão de negócio que não está escrita em lugar nenhum do código — cupons diferentes **podem** acumular. Alguém que olhe só o `aplicar_cupom` não sabe se isso é intencional ou descuido; o teste diz que é intencional.

O quinto é o valor de borda levado ao extremo: cinquenta reais de desconto num pedido de trinta. Sem o `max(0, ...)`, a lanchonete pagaria vinte reais ao cliente.

Exercício 5: as regras que quase morreram ontem

ARQUIVO: TESTES.PY

Este é o exercício mais importante do dia. Cada teste aqui corresponde a um erro do anexo A de ontem — e você vai fazer os três falharem de propósito antes de aceitá-los.

Acrescente aos imports:

```

from personagem import BonusTemporario
from itens import ElixirDeFuria
from classes import Mago, Troll

```

ETAPA 1 · A CLASSE BONUSTEMPORARIO SOZINHA

Crie `TesteBonusTemporario`, com `setUp` criando um `BonusTemporario()` puro — sem personagem nenhum. Três testes:

1. Bônus recém-criado não está ativo.

- Depois de `comecar(6, 3)`: o primeiro `passar_turno()` devolve falso, o segundo devolve falso, o terceiro devolve verdadeiro. Escreva as três afirmações em sequência, no mesmo teste — é a sequência que é a regra.
- Um bônus que nunca começou devolve falso em cinco `passar_turno()` seguidos. Este é o erro 7 do anexo A de ontem.

Repare que estes três testes não sabem que o RPG existe. Ontem você escreveu que a `BonusTemporario` "não sabe a que atributo pertence" — e agora essa frase virou uma coisa que dá para provar: se você conseguiu testar a classe sem criar um `Personagem`, ela realmente não depende de um. **Testes medem o desenho.** Classe difícil de testar sozinha é quase sempre classe amarrada demais em alguma outra.

ETAPA 2 · AS TRÊS REGRAS DO JOGO

Crie `TesteRegra` com quatro testes:

TESTE	COMO ESCREVER
Dois elixires não acumulam	Guarde o ataque original, use dois <code>ElixirDeFuria</code> seguidos, afirme que o ataque subiu exatamente 6 — não 12.
O bônus volta ao normal	Um elixir, depois três <code>iniciar_turno()</code> . O ataque tem que estar igual ao original e <code>bonus_ataque.ativo()</code> tem que ser falso.
A habilidade entra em recarga	Um Guerreiro e um boneco. Use a habilidade, guarde a vida do boneco, use de novo. A vida não pode ter mudado, e <code>habilidade_disponivel()</code> tem que ser falso.
A habilidade volta	Um Mago usa a habilidade e depois passa <code>turnos_de_recarga()</code> turnos. No fim, <code>habilidade_disponivel()</code> verdadeiro. Chame o método em vez de escrever o 4 — assim o teste continua certo se o balanceamento mudar.

ETAPA 3 · FAZER OS TRÊS FALHAREM

Agora a parte que ensina. Um de cada vez, faça a mudança, rode, confira que **exatamente o teste esperado** ficou vermelho, e desfaça antes da próxima:

QUEBRE ASSIM	ERRO DE ONTEM	QUEM TEM QUE FICAR VERMELHO
No <code>aplicar_bonus_ataque_temporario</code> , apague o <code>if self.bonus_ataque.ativo():</code> e o desconto abaixo dele.	Erro 6	1 teste: o dos dois elixires.
No <code>usar_habilidade</code> , apague a linha do <code>iniciar_recarga</code> .	Erro 5	1 teste: o da recarga.
No <code>passar_turno</code> da <code>BonusTemporario</code> , troque o corpo inteiro por <code>return self.turnos == 0</code> .	Erro 7	3 testes. Leia a caixa abaixo.

POR QUE A TERCEIRA QUEBRA DERRUBA TRÊS

Os dois testes da `BonusTemporario` ficam vermelhos porque foi ela que você quebrou. O terceiro — o do bônus voltando ao normal — fica vermelho porque o `Personagem` **depende** dela: com o `passar_turno` mentindo, o ataque nunca é descontado de volta.

Isso é o formato normal de uma falha real. Um defeito numa classe pequena aparece em vários testes de uma vez, e os que ficam vermelhos desenharam o mapa de quem dependia dela. Quando isso acontecer com você, comece consertando pelo teste mais **baixo** — o que testa a classe sozinha. Os de cima costumam ficar verdes junto.

CONFIRA RODANDO

Com o jogo inteiro, `Ran 24 tests` e `OK`. Rode dez vezes seguidas: `OK` nas dez.

Nas três quebras da etapa 3, confira o nome do método na saída, não só a cor. Se você quebrou o `iniciar_recarga` e quem ficou vermelho foi o teste do elixir, alguma coisa não está testando o que você acha que está.

E se uma quebra não deixar nada vermelho? Então essa regra não está sendo testada por ninguém, e o buraco é seu. Escreva o teste que falta antes de desfazer.

A prova final

Se você ainda tem a pasta `rpg-antes/` de ontem — o jogo antes da refatoração — faça isto: copie o seu `testes.py` para dentro dela e rode.

Quase todos vão passar. Os que falharem vão falhar por um motivo só: eles chamam nomes que só existem depois da refatoração (`calcular_dano`, `usar_habilidade`, `bonus_ataque`). Os testes de regra — vida, dano, cura, mochila, elixir que não acumula — passam nos dois.

Isso é o que ontem foi conferido no olho, cinco vezes, em umas duas horas. Hoje leva meio segundo, e leva meio segundo para sempre.

O que você fez hoje

TÉCNICA	O SINAL DE QUE ERA HORA DE USAR
<code>assert</code>	Você conferiu alguma coisa no olho e vai ter que conferir de novo.
<code>unittest</code> e <code>setUp</code>	Passou de meia dúzia de afirmações e uma escondeu as outras.
Afirmar a propriedade	O resultado muda toda vez que roda.
Testar pelo efeito	O método não devolve nada, mas deixa rastro no estado.
Separar decidir de mostrar	A resposta do método sai por <code>print</code> e não dá para pegar de volta.
Teste de regressão	Uma regra que não aparece na tela quando quebra.

O QUE FICA

Teste não é sobre encontrar bugs — encontrar bug é o que ele faz no primeiro dia. O que ele faz nos outros mil dias é te **dar coragem**: coragem de refatorar, de mexer numa classe que você não escreveu, de aceitar a mudança que o cliente pediu na véspera da entrega. Ontem você mudou o jogo com medo, conferindo tudo no olho. Da próxima vez você muda com uma tecla e meio segundo de espera.

Erros que você vai ver

Testar produz uma família de erros própria. Boa parte deles não é o código do jogo estar errado — é o teste estar mal montado, e vale saber distinguir os dois na hora.

1. MODULENOTFOUNDERROR: NO MODULE NAMED 'PERSONAGEM'

```
ModuleNotFoundError: No module named 'personagem'
```

O `testes.py` não está na mesma pasta dos outros quatro arquivos, ou você rodou o comando de outra pasta. Entre na pasta do jogo e rode `python testes.py` ali dentro. Confira com `dir` (ou `ls`) que os cinco arquivos aparecem juntos.

2. RAN 0 TESTS

```
-----  
Ran 0 tests in 0.000s
```

```
OK
```

O `OK` aqui é uma mentira gentil: zero testes falharam porque zero testes rodaram. Três causas, em ordem de frequência: o método não começa com `test_`; a classe não herda de `unittest.TestCase`; ou faltou o bloco `if __name__ == "__main__": unittest.main()` no fim do arquivo.

Olhe o número de testes toda vez que rodar. Se ele cair sem você ter apagado nada, algum teste sumiu do radar.

3. ATTRIBUTEERROR NUM ATRIBUTO QUE O SETUP DEVERIA TER CRIADO

```
AttributeError: 'TesteVida' object has no attribute 'heroi'
```

Você escreveu `setup`, `Setup` ou `set_up`. O nome é `setUp`, com U maiúsculo e mais nada. Escrito diferente, ele vira um método comum que ninguém chama, e o `self.heroi` nunca é criado.

4. O TESTE TRAVA E NÃO TERMINA NUNCA

Sem erro, sem saída, o cursor piscando para sempre.

Alguns caminhos do teste chegaram num `input`. Provavelmente você chamou `turno_do_jogador`, `abrir_mochila` ou `escolher_personagem`. Aperte `Ctrl+C` e teste o método que faz a conta, não o que conversa com o jogador.

5. TYPEERROR: '<' NOT SUPPORTED BETWEEN INSTANCES OF 'NONETYPE' AND 'INT'

```
TypeError: '<' not supported between instances of 'NoneType' and 'int'
```

Você comparou o retorno de um método que só imprime. `atacar`, `defender` e `subir_nivel` devolvem `None`, sempre. É o capítulo 4: teste o efeito no estado, não o retorno.

6. O TESTE PASSA ÀS VEZES E FALHA ÀS VEZES

Rodou cinco vezes: quatro `OK` e um `FAILED`, sem você ter tocado em nada.

Sorteio não domado — capítulo 3. Alguma afirmação sua depende do valor sorteado. Troque por uma afirmação sobre a faixa, ou aumente o número de repetições até a coincidência ficar impossível, ou fixe a semente se você precisa de um caminho específico.

Um teste instável tem que ser consertado ou apagado no mesmo dia. Deixá-lo vermelho de vez em quando estraga todos os outros.

7. UM TESTE PASSA SOZINHO E FALHA JUNTO COM OS OUTROS

Rodando só ele, `OK`. Rodando o arquivo inteiro, `FAIL`.

Estado vazando entre testes. Ou você criou o objeto fora do `setUp` — no corpo da classe, o que faz ele ser um só para todos os métodos — ou está mexendo numa constante do módulo e não desfazendo. Cada teste tem que montar o próprio mundo do zero.

Lembre também que o `unittest` roda os métodos em ordem alfabética, não na ordem em que estão escritos. Se a ordem importa para o seu teste, o teste está errado.

8. ATTRIBUTEERROR: MODULE 'RANDOM' HAS NO ATTRIBUTE 'SEED'

```
AttributeError: module 'random' has no attribute 'seed'
```

Você tem um arquivo chamado `random.py` na pasta. O Python acha o seu antes do dele, e o seu não tem `seed`. Renomeie o arquivo e apague a pasta `__pycache__`.

Vale para qualquer nome da biblioteca padrão: `unittest.py`, `itens.py` não dá problema, mas `math.py`, `json.py` e `time.py` dão.

9. O TESTE FALHOU E VOCÊ "CONSERTOU" O TESTE

Sem erro na tela. Este é o único erro do anexo que é de disciplina, não de sintaxe.

O teste dizia que o ataque tinha que subir 6, subiu 12, e você trocou o 6 por 12 no teste. Pronto, ficou verde. E a regra do jogo foi embora.

Quando um teste fica vermelho, a primeira pergunta é sempre **"quem está errado?"**. Se a regra mudou de propósito, o teste é que tem que mudar — e aí você muda entendendo o que está abrindo mão. Se a regra não mudou, o código é que está errado. Mudar o teste sem responder essa pergunta transforma o arquivo inteiro em enfeite.

Extras

Tudo aqui é opcional. Nenhum capítulo depende destes itens.

E1.1 A MENSAGEM QUE AJUDA

Depende de: capítulo 1.

Pegue três dos seus `assert` do capítulo 1 e escreva as mensagens de três jeitos: sem mensagem nenhuma, com uma mensagem genérica ("deu errado"), e com uma mensagem que descreve a regra e mostra o valor obtido — `f"curar demais deveria parar em {heroi.vida_maxima}, mas deu {heroi.vida}"`.

Quebre o código e leia as três saídas. Responda em três linhas: quanto tempo cada uma economizaria de você daqui a seis meses?

E2.1 O RELATÓRIO DETALHADO

Depende de: capítulo 2.

Rode `python testes.py -v` e compare com a saída normal. Depois rode `python -m unittest testes -v` e `python -m unittest testes.TesteVida`.

Responda: para que serve rodar uma classe de teste sozinha, e em que situação do seu dia isso economizaria tempo?

E2.2 O NOME COMO DOCUMENTAÇÃO

Depende de: capítulo 2.

Rode com `-v` e leia só a coluna dos nomes dos métodos, sem olhar o código. Dá para entender as regras do jogo?

Renomeie os que não passarem nesse teste. Um bom nome de teste é uma frase:

`test_curar_nao_passa_da_vida_maxima`, não `test_curar_2`. Compare o seu arquivo com o anexo C e veja se o checklist ficou redundante.

E3.1 INJETAR O DADO

Depende de: capítulo 3.

Faça o `atacar` aceitar um parâmetro `rolagem=None`, sorteando só quando ele vier vazio. Escreva dois testes sem semente nenhuma: um passando `rolagem=1` (erro garantido) e outro passando `rolagem=20` (acerto garantido).

Agora julgue: o jogo ficou melhor ou só ficou mais testável? Escreva cinco linhas. Não existe resposta certa — existe a sua, defendida.

E4.1 CALAR O JOGO DURANTE OS TESTES

Depende de: capítulo 4.

Procure `contextlib.redirect_stdout` e `io.StringIO` na documentação. Faça um `setUp` que desvie o `print` para dentro de uma variável, e um `tearDown` que devolva ao normal.

Depois use isso para escrever um teste que afirma o **texto**: que o Guerreiro imprime "atacou sem falhar" ao atacar. Responda: esse teste é bom ou é uma corda no pescoço?

E5.1 O QUE NINGUÉM TESTA

Depende de: capítulo 5.

Abra o `personagem.py` e o `itens.py` e faça uma lista de todos os métodos públicos. Marque quais têm pelo menos um teste.

Isso se chama **cobertura**, e existem ferramentas que calculam sozinhas. Responda: dos métodos sem teste, quais valem um teste e quais não valem? O que faz um método não valer?

E5.2 OS TESTES RODANDO SOZINHOS

Depende de: capítulo 5.

Este repositório já roda uma verificação a cada envio: o arquivo `.github/workflows/verificar.yml` confere a sintaxe de todos os `.py`. Leia esse arquivo e entenda cada linha.

Responda no papel: o que precisaria mudar nele para rodar `python -m unittest` também? E qual seria a consequência prática de um teste quebrado passar a barrar o envio?

E5.3 LEITURA

Depende de: capítulo 5.

Procure os termos **teste unitário**, **teste de integração** e **TDD** (*test-driven development*).

Responda: os testes que você escreveu hoje são unitários ou de integração? O

`test_o_goblin_se_defende_com_pouca_vida` é o mesmo tipo que o `test_curar_nao_passa_da_vida_maxima`? E a ordem do capítulo 5 — teste vermelho antes do conserto — tem a ver com qual dos três termos?

Checklist de entrega

O ARQUIVO

- ☐ Existe `testes.py` na mesma pasta dos outros quatro arquivos.
 - ☐ Nenhum dos outros quatro arquivos importa o `testes.py`.
 - ☐ `python testes.py` roda sem erro e termina em `OK`.
 - ☐ O número de testes é 24 ou mais.
 - ☐ Todo método de teste começa com `test_` e tem nome de frase.
 - ☐ Nenhum teste chama `input`.
-

CAPÍTULOS 1 E 2

- ☐ As cinco regras de vida e dano têm um teste cada.
 - ☐ Existe o teste da defesa cortando o dano pela metade.
 - ☐ As classes de teste herdam de `unittest.TestCase`.
 - ☐ Existe `setUp`, e nenhum objeto de teste é criado fora dele.
 - ☐ O `Inventario` tem teste de vazio, de adicionar, de tirar e de número inválido.
 - ☐ Não sobrou nenhum `assert` cru no arquivo.
-

CAPÍTULOS 3 E 4

- ☐ O teste da faixa do dano importa `VARIACAO_DO_DANO` em vez do número 3.
 - ☐ O arquivo roda dez vezes seguidas sem nenhuma falha intermitente.
 - ☐ Existe um teste que prova que o Guerreiro nunca erra.
 - ☐ Existe um teste que usa semente para forçar um ataque errado.
 - ☐ Existe pelo menos um teste que afirma sobre o estado depois da ação, e não sobre o retorno.
 - ☐ Existe um teste sobre a decisão de um inimigo.
-

CAPÍTULO 5

- ☐ A `BonusTemporario` é testada sem criar nenhum `Personagem`.
 - ☐ Existe o teste da sequência falso, falso, verdadeiro do `passar_turno`.
 - ☐ Existe o teste de que dois elixires seguidos não acumulam.
 - ☐ Existe o teste de que a habilidade entra em recarga e volta depois.
 - ☐ Nenhum teste lê `bonus_ataque.turnos` ou `bonus_ataque.quantidade` direto.
 - ☐ As três quebras propositalis da etapa 3 foram feitas, e cada uma deixou vermelho o teste esperado.
-

O ÚLTIMO TESTE

ANTES DE ENTREGAR

Escolha um teste qualquer do seu arquivo. Vá no jogo e quebre a regra que ele testa. Rode.

Se ficar vermelho, o seu arquivo de testes está vivo. Se ficar verde, ele é enfeite — e é melhor descobrir isso agora do que na próxima vez que você precisar mexer no jogo com pressa.

Desfaça a quebra. Rode uma última vez. .